

[Congestion Control =] בקרת עומסים

מעבר מבקרת זרימה (Flow Control) לבקרת עומסים (Congestion Control) הוא המעבר מהסתכלות "נקודתית" (האם יש מקום בזיכרון של הנמען?) להסתכלות "מערכתית" (האם הנתיב שבו הנתונים עוברים מסוגל להעביר אותם בלי לקרוס?).

מהי בקרת עומסים (Congestion Control)?

הגדרה: בקרת עומסים היא סט של אלגוריתמים המנוהלים על ידי ה-TCP Sender, שמטרתם למנוע מצב שבו הרשת (הנתבים והמתגים שבין TCP השולח לבין היעד) הופכת ל"פקוקה".

למה זה קריטי?

אם רשת הופכת לעמוסה, המתגים (= switches) והנתבים (= routers) מתחילים לזרוק חבילות (Packet Drops) כי התורים שלהם מלאים. ללא בקרת עומסים, כל מחשב ימשיך לשלוח נתונים באותה 'עוצמה', מה שיוביל ל- **Congestion Collapse** – מצב שבו אף אחד לא מקבל נתונים, והרשת משקיעה את כל כוחה בשידור חוזר של חבילות שאבדו.

הפרמטר cwnd (Congestion Window)

זהו הפרמטר החשוב ביותר במכונת ה-TCP. בעוד שה-rwnd מייצג את המגבלה של הנמען, ה-cwnd מייצג את המגבלה של הרשת.

- **מה הוא cwnd:** מספר הבתים (או הסגמנטים) ש-TCP השולח רשאי להחזיק "באוויר" (In-flight) מבלי לקבל עליהם אישור (ACK).
- **החוק הקריטי:** השולח יכול לשלוח בכל רגע נתון כמות נתונים שאינה עולה על: $\text{Min}(\text{rwnd}, \text{cwnd})$

מהו ערכו ההתחלתי?

- בעבר, הערך היה סגמנט אחד (1 MSS). כיום, בתקנים מודרניים (כמו ב-Linux 6.14), הערך ההתחלתי (Initial Window) הוא 10 MSS (לרוב כ- 14.6KB).
- **למה 10?** זו פשרה: זה מאפשר ל-HTTP GET טיפוסי לעבור ב"נסיעה אחת" (Round Trip) מבלי להמתין לאישורים, ומצד שני זה מספיק קטן כדי לא להפיל רשתות 'חלשות'.

איך ה-cwnd משתנה? (הדינמיקה)

האלגוריתם מתנהג כ"בלש" שמנסה לגלות כמה רוחב פס פנוי יש ברשת:

1. **Slow Start (השלב האגרסיבי):** בתחילת החיבור, ה-cwnd מוכפל בכל פעם שמתקבל ACK.
○ הגיון: "אני לא יודע כמה מהר אני יכול לרוץ, אז בואו נכפיל את המהירות בכל פעם שמישהו מאשר לי שהכל בסדר". זהו גידול מעריכי (Exponential).
2. **Congestion Avoidance (שלב הדיוק):**
ברגע שה-cwnd מגיע לסף מסוים (sssthresh), המערכת עוברת לגידול ליניארי (הוספת 1 MSS לכל RTT).
○ הגיון: "הגעתי למהירות גבוהה, בואו נבדוק בזהירות את הגבול".

4. דוגמה מספרית (Slow Start)

נניח Initial CWND = 10 MSS (נתעלם מ-rwnd כרגע):

- **RTT 0:** השולח משדר 10 סגמנטים.
- **RTT 1:** השולח מקבל 10 ACKs. כל ACK מאשר העברה מוצלחת. המערכת מגדילה את ה-cwnd ב-10. כעת ה-cwnd = 20 MSS.
- **RTT 2:** השולח משדר 20 סגמנטים. מקבל 20 ACKs. ה-cwnd הופך ל-40.

בקרת העומסים אינה סטאטית.

היא מערכת תגובתית (Reactive) שצריכה להבחין בין "רשת איטית" לבין "רשת עמוסה".

1. ה-ssthresh (Slow Start Threshold): הבלם המקדים

ה-ssthresh הוא ערך קריטי שמהווה את "נקודת המעבר" בין שלב הגידול המעריכי (Slow Start) לבין שלב הגידול הליניארי (Congestion Avoidance).

- **איך הוא נקבע?** בדרך כלל, הוא נקבע לערך שבו המערכת זיהתה עומס בפעם האחרונה. המערכת אומרת: "עד כאן הגעתי בשלום, מעבר לזה – בוא נתקדם בזהירות".
- **מקרה קצה:** אם החיבור חדש לגמרי, ssthresh מוגדר כערך אינסופי (או מקסימלי), כדי לאפשר למערכת לטוס כמה שיותר מהר עד שהיא פוגשת את ה-"Wall" של הרשת.

2. כשדברים משתבשים: תגובה ל-Packet Loss

זהו הרגע שבו ה-cwnd משנה את פניו. בלינקס 6.14, הזיהוי של אובדן מנות מתבצע בשתי דרכים עיקריות:

א. Triple Duplicate ACKs (אובדן קל)

הלקוח מקבל ACK אחד, ואחריו שלושה אישורים זהים לאותו מספר רצף.

- **הפרשנות:** ה-Receiver אומר: "קיבלתי את מנה X, אבל אני ממשיך לקבל מנות שבאו אחרי מנה Y שאבדה".

- **תגובת ה-cwnd (Fast Retransmit):** המערכת לא עוצרת. היא מורידה את ה-cwnd לערך נמוך (לרוב חצי מהערך הנוכחי) וממשיכה לרוץ. זהו מצב של "האטה מבוקרת".

- ב. RTO Timeout (אובדן חמור)**
הטיימר שדיברנו עליו קודם פקע. לא הגיע שום ACK.
- **הפרשנות:** "הקשר נותק או שהרשת עמוסה עד כדי קריסה".
 - **תגובת ה-cwnd (Hard Reset):** המערכת מבצעת את הצעד הכי קיצוני: היא מאפסת את ה-cwnd ל-1 MSS. היא חוזרת לנקודת ההתחלה של ה-Slow Start. זהו ה"פחד" של TCP – הוא מעדיף להיות איטי מאוד מאשר להחמיר את העומס על הרשת.

דוגמאות למקרי קצה (Edge Cases)

תרחיש	תגובת ה-cwnd	הסבר הנדסי
רשת אלחוטית (רעש)	ירידה חדה	TCP מפרש שגיאת ביטים כעומס וחוזר ל-1 MSS. זו "הטרגדיה של TCP" מעל אלחוט.
Bufferbloat	גדילה לינארית	הנתב בדרך מחזיק תור ענק. ה-cwnd ממשיך לגדול עד שהתור מתמלא, מה שגורם ל-Latency להזנק.
קישור אופטי (יציב מאוד)	גדילה מתמדת	ה-cwnd עשוי להגיע למאות מגה-בייטים. כאן נכנס לתמונה ה-Window Scaling.

Linux 6.14 ו-BBR: המהפכה

- בלינוקס 6.14, ייתכן מאוד שלא נשתמש בשיטת ה"חציצה" (חצי מה-cwnd ב-Loss).
מנוע ה-BBR [Bottleneck Bandwidth and Round-trip propagation time] של גוגל, שמיושם בלינוקס, עובד אחרת:
1. הוא מודד את קצב ההעברה המקסימלי (Bandwidth) של הנתב.
 2. הוא מודד את ה-RTT המינימלי.
 3. הוא מחשב את ה-cwnd האידיאלי: $cwnd = Bandwidth * RTT$.
- **היתרון:** הוא לא מחכה ל-Packet Loss כדי להוריד מהירות. הוא פשוט שומר על החלון בגודל שבו הצינור מלא, אבל לא גדוש.

המעבר מ-Slow Start ל-Congestion Avoidance הוא הרגע שבו ה-TCP עובר ממצב של "חיפוש אגרסיבי" למצב של "שמירה על יציבות".

רשתות תקשורת, תשפ"ו
Congestion Control
עמוד 3

זהו המנגנון שמונע מהרשת לקרוס ברגע שהגענו לקיבולת שלה.

ההבדל הקונספטואלי

- **Slow Start (האצה):** המטרה היא "לגלות" את רוחב הפס הפנוי במהירות. בגלל שכל ACK מגדיל את ה-cwnd ב-1 MSS, הגידול הוא **אקספוננציאלי**.
- **Congestion Avoidance (התמדה):** ברגע שהגענו ל-ssthresh (הסף), אנחנו עוברים לגידול **ליניארי**. המטרה היא "לדגום" את רוחב הפס בזהירות, תוספת קטנה בכל פעם, כדי לא לגרום ל-Packet Loss פתאומי.

המנגנון: איך זה עובד מתמטית?

- במצב Congestion Avoidance, עבור כל RTT (זמן הלך-חזור) שבו כל הסגמנטים אושרו בהצלחה, ה-cwnd גדל ב-1 MSS **בלבד**.
- **ההיגיון:** אם ה-cwnd שלנו הוא כרגע 20 MSS, ב-RTT הבא הוא יהיה 21 MSS, אחר כך 22, וכן הלאה.
 - **השוואה:** אם ב-Slow Start היינו מגיעים מ-20 ל-40 ב-RTT אחד, ב-Congestion Avoidance ייקח לנו **20 RTT** להגיע ל-40. הגידול הפך איטי פי כמה, וזה בדיוק מה שמונע "פיצוץ" של תורים בנתבים.

3. מתי עוברים מהאחד לשני? (ה-ssthresh)

המעבר מתבצע ברגע ש-cwnd = ssthresh.

- **תרחיש:** נניח $\text{ssthresh} = 16 \text{ MSS}$.
- ה-cwnd גדל ב-Slow Start: 1, 2, 4, 8, 16.
- ברגע שהגענו ל-16, המערכת מזהה שהגענו ל-"אזור הסכנה" (הערך שהוגדר כסף $= \text{threshold}$).
- מהרגע הזה והלאה, ה-cwnd יגדל ב-1, לא ב-16.

4. האם זה מספיק כדי למנוע עומס? (התשובה היא לא!)

זהו החלק שחשוב לסטודנטים להבין: Congestion Avoidance אינו מונע עומס, הוא רק מאט את קצב הגדילה שלו.

בסופו של דבר, בשיטה הליניארית הזו, ה-cwnd ימשיך לגדול עד שיהיה עומס ברשת (Packet Loss). אז המערכת תזהה את האובדן, תכווץ את ה-cwnd (בדרך כלל לחצי), ותעדכן את ה-ssthresh לערך החדש (חצי מהערך שגרם לאובדן).

התהליך הזה יוצר את ה-Sawtooth (שן מסור):

1. עלייה ליניארית: אנחנו בודקים את גבולות הרשת.
2. ירידה חדה: זיהינו אובדן, הורדנו מהירות.
3. חזרה לעלייה: המערכת מנסה שוב.

הקשר ל-Linux 6.14 ולמודרניזציה

בקרנלים מודרניים, ה-Congestion Avoidance ה"קלאסי" (מבוסס Reno/Cubic) הופך לפחות ופחות רלוונטי בגלל בעיית ה-Bufferbloat.

- בלינוקס 6.14, הגישה החדשה היא BBR (Bottleneck Bandwidth and RTT).

- במקום לחכות ל-Packet Loss כדי להוריד מהירות (שזה לב העיקרון של ה-Congestion Avoidance הישן), ה-BBR מודד את ה-RTT המינימלי ואת ה-Bandwidth המקסימלי ומנסה להחזיק את ה-cwnd בערך האופטימלי לפני שהתורים בנתבים מתמלאים.

האבולוציה של בקרת העומסים (Congestion Control)

האבולוציה של בקרת העומסים (Congestion Control) היא סיפור של "התפכחות". בתחילת הדרך, TCP היה "אופטימיסט נאיבי" שחשב שהרשת היא משאב אינסופי, ועם השנים הוא הפך ל"מנהל סיכונים" מתוחכם שמשמש בסטטיסטיקה כדי לחזות עומס לפני שהוא קורה.

להלן הסקירה ההיסטורית, מהתמימות ועד ל-BBR:

1. עידן התמימות (1974 - 1986): "פשוט תשלח"

- **המצב:** TCP המקורי (RFC 793) לא כלל שום מנגנון בקרת עומסים. הוא הסתמך רק על ה-rwnd (בקרת זרימה) של הנמען.
- **הקריסה (1986):** האינטרנט (אז ה-ARPANET) חווה **Congestion Collapse**. התעבורה בנתיבים מסוימים צנחה מ-32 Kbps ל-40 bps. השולחים פשוט הציפו את הנתבים בשידורים חוזרים (Retransmissions) כי הם לא ידעו שהנתבים זורקים את החבילות בגלל עומס, ולא בגלל תקלות.

2. המהפכה של ואן ג'ייקובסון (1988): המעבר למדע

- **האירוע:** ואן ג'ייקובסון פרסם את המאמר המכונן "*Congestion Avoidance and Control*".
- **החידוש:** הוא הציג את ה-Slow Start ואת ה-Congestion Avoidance (הגידול הליניארי).
- **המשמעות:** TCP הפך ל-"Conservation of Packets" – הוא למד "לשמור" על הרשת. הוא לא היה רשאי להכניס חבילה חדשה לרשת, אלא אם כן יצאה חבילה ישנה (כלומר, הגיע ACK). זהו עקרון ה-Self-Clocking (הרשת מתזמנת את הקצב של השולח).

3. עידן ה-Loss-Based (שנות ה-90 עד ה-2000)

- אלגוריתמים כמו Reno, TCP Tahoe ו-NewReno שלטו בכיפה.
- **הלוגיקה:** "אני מניח שהכל תקין כל עוד אין Packet Loss".
- **השיטה:** ברגע שיש Loss, אני מוריד את ה-cwnd לחצי (Multiplicative Decrease).
- **הבעיה:** זו גישה "תגובתית" (Reactive). אתה חייב "לפגוע בקיר" כדי לדעת שהוא שם. זה יצר רעידות (Oscillations) ברוחב הפס – הרשת אף פעם לא הייתה יציבה.

4. עידן ה-Delay-Based (אמצע שנות ה-2000): "לפני הקיר" אלגוריתמים כמו TCP Vegas.

- הלוגיקה: "אני אזהה עומס לפי עלייה ב-RTT (זמן הלוך-חזור)".
- התובנה: ככל שהנתב בדרך עמוס יותר, התור שלו מתארך, ולכן לוקח למנה יותר זמן לעבור.
- ההצלחה/כישלון: רעיון מבריק, אבל בגלל שהרשתות היו מעורבות (רעש פיזי באלחוט שגרם ל-RTT להשתנות סתם כך), Vegas לא הצליח להחליף את Reno/Cubic.

5. עידן ה-CUBIC (2000 - 2015): High-Speed

- המצב: רוחב הפס עלה דרמטית, ו-Reno/NewReno לא הצליחו להתאושש מהר מספיק אחרי אובדן מנה.
- הפתרון: CUBIC (ברירת המחדל של לינוקס במשך שנים).
- השיטה: במקום להגדיל את ה-cwnd בצורה ליניארית (פלוס 1 כל RTT), הוא משתמש בפונקציה קובית. ככל שהשולח רחוק מהאובדן האחרון, הוא גדל יותר מהר; ככל שהוא מתקרב לאזור האובדן, הוא מאט.

6. עידן ה-Modelling & Modern AI (2016 - היום): BBR

- המהפכה של גוגל (BBR - Bottleneck Bandwidth and RTT):
- הלוגיקה: "עזוב אותי מה-Packet Loss, עזוב אותי מהעלייה ב-RTT. אני אמדוד מהו הקצב המקסימלי האמיתי שהצינור מסוגל להעביר (Bottleneck Bandwidth)".
- התפיסה: BBR לא מחכה שיהיה Loss. הוא שומר על הצינור מלא בדיוק לפי ה-BDP (Bandwidth-Delay Product) האופטימלי. זה מאפשר ל-YouTube ולשירותי הענן של גוגל להעביר נתונים במהירות עצומה גם על רשתות "רועשות".

סיכום התפתחותי

1. התקופה הפרה-היסטורית: "תשלח הכל, מקסימום יפול".
2. תקופת ה-Jacobson: "תמדוד, תהיה זהיר, אל תציף".
3. תקופת ה-Loss-Based: "תוריד חצי בכל פעם שיש אובדן" (ריאקטיבי).
4. תקופת ה-BBR: "תמדוד את היכולת של הצינור ותעבוד בשיא היעילות" (פרואקטיבי).

הסקירה הזו ממחישה איך עברנו ממערכת שהיא "מגיבה לכישלונות" למערכת שהיא "מנהלת ביצועים".